Your grade: 100%

Your latest: 100% • Your highest: 100% • To pass you need at least 80%. We keep your highest score.

Next item

Assignment Introduction

As you know, one important aspect of programming is testing. Poker hand evaluation is fairly complex, and requires a lot of test cases. We won't have you write a completely comprehensive set of test cases here, but we want you to start thinking about what is involved in testing this code. We also want you to think about a variety of things that could go wrong, so that you can avoid those mistakes when you write the hand evaluation logic in the next lab.

For each of the following questions, you will be presented with a reason to write a particular test case: either coverage of some particular outcome, or a mistake that a programmer might make. We have made a correct and broken implementation of the poker hand evaluation logic. For the "description of a bug" test case motivations, the broken implementation has the particular bug described. For the "coverage of a case" motivations, the broken implementation does not correctly handle the case that was described. Your goal is to write a test case which shows the bug (or covers the case) for each of these.

To write a test case, you will write a single call to the "check" function which takes one parameter: the input hand description, as could be passed to your hand_from_string function from the poker_input lab. For example, if you thought that the poker hand "As Kc 9d 4h 3c 2d" would show the bug, you would write

```
check( "As Kc 9d 4h 3c 2d" )
```

Note that our hand evaluation logic will sort the cards before evaluating the hand. Also, you must give a card with 5, 6, or 7 cards, as we will constrain hand evaluation logic to only need to be correct on those hand sizes.

1. One important aspect of testing is making sure that you cover all the expected behaviors of the code. Write a test case which covers the case of a flush in clubs: — 1 / 1 point

```
1  check("Ac Kc 9c 4c 3c")
```

Run Reset

Correct
Good job!

2. Suppose the programmer had an off-by-one bug in their code when counting the cards for a flush, and accidentally considers 4 cards of the same suit to be a flush. Write a test case which catches this bug: — 1 / 1 point

```
1  check("Ac Kc 9c 4c 2s")
```

Run Reset

Correct
Good job!

3. Write a test case which covers the case of a four-of-a-kind hand. — 1 / 1 point

```
1  check("Ac As Ah Ad 2s")
```

Run Reset

Correct
Good job!

4. One common source of bugs is programmers incorrectly understanding the specification. In poker, one possible misunderstanding is the assumption that if a hand has a straight AND a hand has a flush, then the hand has a straight flush. Write a test case which catches this bug. Note that this assumption is correct if the hand has exactly 5 cards: you will need to write a test case with more than 5 cards for this mistake. — 1 / 1 point

```
1  check("As Kc Qc Jc 0c 2c")
```

Run Reset

Correct
Good job!

5. Suppose the programmer does not properly detect full houses (e.g., declares them to be three of a kind). Write a test case to catch this mistake: — 1 / 1 point

```
1  check("Ac As Ad 4c 4s")
```

Run Reset

Correct
Good job!

6. The hand evaluation needs to not only state the ranking (pair, flush, etc) of a hand, but also give the proper 5 cards which make it up (and in the proper order for tie breaking). Suppose the programmer did not think carefully about this for hands that are "two pairs" : they correctly place the two pairs into the first two places in the hand, but do not find the proper tie breaking card and put it into the 5th position in the hand. Write a test case which catches this bug: — 1 / 1 point

```
1  check("Kc Ks 9h 9d As")
```

Run Reset

Correct
Good job!

7. Two types of hands (full house and two pairs) require finding a "secondary pair"---another pair in the hand which is not the same as the three of a kind (full house) or highest-valued pair (two pair). The secondary pair should be the **highest value** pair which is distinct from the three of a kind or primary pair. Suppose the programmer iterates through the hand like this (psuedo-code, not real code): — 1 / 1 point

```
secondary_pair = None
for c in hand:
      if c is part of a pair:
            secondary_pair = c.value
            pass
return secondary_pair
```

This code might give a secondary pair which is not the highest valued one in the hand. Write a test case which catches this bug:

Note: you will need to write a test case that uses 7 cards.

```
1  check("Ac As 9c 9d 2s 2h")
```

Run Reset

Correct
Good job!

8. One unusual thing about Poker hands is that an Ace can be either high or low (but not both at the same time) when determining if a hand has a straight. Suppose the programmer did not consider the fact that Aces can be low when determining if a hand has a straight, and write a test case which catches this mistake. — 1 / 1 point

```
1  check("5c 4c 3c 2s Ad")
```

Run Reset

Correct
Good job!

9. Another tricky situation in picking the right 5 cards for the final hand arises in straights. If the hand only has 5 cards, the programmer can safely assume that if there is a straight, the cards that make it up are consecutive in the hand. However, with more than 5 cards in a hand, it is possible to have a straight with other cards in between the ones that make up a straight (even if the cards are sorted). Write a test case which catches the mistake of a programmer incorrectly assuming that the 5 cards for a straight are all consecutive (after sorting) without any other cards in between. — 1 / 1 point

Note: You will need to use larger than a 5 card hand for this test case.

```
1  check("Ac Kc Qc Qc Js 0h")
```

Run Reset

Correct
Good job!

10. Suppose the programmer had an off-by-one bug in their logic to check for straights and only requires 4 cards in a row rather than 5. Write a test case that catches this mistake: — 1 / 1 point

```
1  check("Ac Kc Qc Jc 2s")
```

Run Reset

Correct
Good job!

11. Checking for an Ace-low straight requires finding an Ace, then finding a 4 card straight of 5, 4, 3, 2 later in the hand. Suppose the programmer only checked for an Ace and then a 4 card straight later in the hand, but did not constrain the 4 card straight to start with 5. Write a test case to catch this mistake. — 1 / 1 point

```
1  check("Ac 6c 5d 4c 3s")
```

Run Reset

Correct
Good job!